

华东师范大学数据科学与工程学院实验报告

课程名称: AI 基础	年级: 2024	上机实践时间: 2026.04.11
指导教师: 杨彬	姓名: 孙育泉	
上机实践名称: Maze	学号: 10234900421	

目录

I 实验任务	1
II 使用环境	1
III 实验过程	1
III.1 基础封装	1
III.2 迷宫生成	2
III.3 搜索算法	5
III.4 可视化渲染	11
III.5 Benchmark	12
IV 总结	13
V 附录	13

I 实验任务

给定一个 $n \times m$ 的二维整数数组，用来表示一个迷宫，数组中只包含 0 或 1，其中 0 表示可以走的路，1 表示不可通过的墙壁。最初，有个人位于左上角 $(1, 1)$ 处，已知该人每次可以向上、下、左、右任意一个方向移动一个位置。

请问，该人从左上角移动至右下角 (n, m) 处最少需要移动多少次。数据保证 $(1, 1)$ 与 (n, m) 处的值均为 0，且至少存在一条路径可以到达。并且需要进行可视化。

考虑使用这些算法：DFS, BFS, Dijkstra, A*。

II 使用环境

编程语言：Python 3

III 实验过程

III.1 基础封装



Idea

为了代码的可维护性与解藕性，这里对代码进行了很大的改动，进行了模块化。

首先，为了进行后续的可视化与维护每次探索的状态，我们将迷宫封装成了一个 `Maze` 类，并且定义了一个 `SolverState` 数据类来保存每一步的快照，方便后续的可视化展示。

py

```
1 # maze.py
2 from dataclasses import dataclass
3 from typing import List, Tuple, Set
4
```

```

5
6 @dataclass
7 class SolverState:
8     """用于寻路算法与可视化解耦的数据模型，保存每一步的快照"""
9
10     current: Tuple[int, int]
11     explored: List[Tuple[int, int]]
12     frontier: List[Tuple[int, int]] # 待探索队列（栈/队列/优先队列）
13     path: List[Tuple[int, int]]
14
15
16 class Maze:
17     """迷宫的基础数据结构"""
18
19     def __init__(
20         self,
21         grid: List[List[int]],
22         start: Tuple[int, int] = (0, 0),
23         goal: Tuple[int, int] = None,
24     ):
25         self.grid = grid
26         self.rows = len(grid)
27         self.cols = len(grid[0])
28         self.start = start
29         self.goal = goal if goal else (self.rows - 1, self.cols - 1)
30
31     def is_wall(self, r: int, c: int) → bool:
32         return self.grid[r][c] == 1
33
34     def is_valid_move(self, r: int, c: int) → bool:
35         return 0 ≤ r < self.rows and 0 ≤ c < self.cols and not
36             self.is_wall(r, c)
37
38     def get_neighbors(self, r: int, c: int) → List[Tuple[int, int]]:
39         directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
40         neighbors = []
41         for dr, dc in directions:
42             nr, nc = r + dr, c + dc
43             if self.is_valid_move(nr, nc):
44                 neighbors.append((nr, nc))
45         return neighbors

```

III.2 迷宫生成

然后，我们需要生成迷宫，用于探究算法在不同的迷宫类型下的性能表现。为此，我们定义了一个 `MazeGenerator` 类，包含多种不同拓扑结构的迷宫生成策略。每种生成方法都保证生成的迷宫至少存在一条解，并且返回一个 `Maze` 类的实例，方便后续的寻路算法使用。

py

```

1 # generators.py
2 import random
3 from collections import deque
4 from maze import Maze
5
6
7 class MazeGenerator:

```

```

8      """
9      迷宫生成器：包含多种不同拓扑结构的迷宫生成策略。
10     所有生成方法均返回 maze.Maze 类的实例。
11     """
12
13     @staticmethod
14     def _is_solvable(grid: list[list[int]]) → bool:
15         """内部辅助函数：使用轻量级 BFS 快速验证网格是否至少存在一条解"""
16         ...
17
18         ...

```

接下来，介绍一下这几种迷宫生成方法：

- (1) `generate_random_density`：生成**随机密度迷宫**（如 图 1 所示），墙壁的分布完全随机，但保证至少存在一条解。

- **原理**：算法遍历网格中的每一个格子，并生成一个 0 到 1 之间的随机数。如果该数字小于设定的阈值，则将该位置设为障碍物，否则设为平地。由于纯随机可能导致起点和终点被彻底封死，每次生成后使用轻量级 BFS 快速跑一遍，如果无解就直接丢弃并重新生成，直到刷出一张有解的地图。
- **地图特点**：毫无章法、高度碎片化。它不存在传统意义上的“走廊”或“死胡同”，而是布满了大小不一的开阔空地和随机散落的像素块。
- **接口**：

py

```

1 @staticmethod
2 def generate_random_density(
3     rows: int, cols: int, wall_probability: float = 0.25
4 ) → Maze:
5     """生成随机密度的迷宫（保证有解）"""
6     ...

```

- (2) `generate_perfect_maze`：生成**完美迷宫**（如 图 2 所示），即没有环路的迷宫，所有死胡同都只有一个出口。

- **原理**：算法模拟了一个“不撞南墙不回头”的矿工。从起点开始，随机选择一个相邻且未被打通的格子，挖穿中间的墙，并把新位置压入栈中。如果走到一个周围所有格子都已经被挖过的地方（死胡同），就从栈中弹出上一个位置进行“回溯”，直到找到新的未开发分支，继续往下挖。
- **地图特点**：高曲折度，长且深的死胡同。它拥有大量的分叉和转弯，使得路径错综复杂，但每条死胡同都只能通向一个出口。
- **接口**：

py

```

1 @staticmethod
2 def generate_perfect_maze(rows: int, cols: int) → Maze:
3     """使用 Randomized DFS 生成完美迷宫（高曲折度，长且深的死胡同）"""
4     ...

```

- (3) `generate_braid_maze`：生成**编织迷宫**（如 图 3 所示），即多解迷宫，存在环路。

- **原理**：它是在完美迷宫的基础上进行二次加工的产物。算法遍历已经生成好的完美迷宫，找出所有三面环墙的“死胡同”。然后，根据设定的概率，随机打通死胡同的一堵墙，使其与相邻的另一条道路强行连通。这样就形成了环路，增加了多解的可能性。

- **地图特点:** 迷宫中充满了“环路”，基本上消灭了死胡同.这意味着从起点到终点存在无数种不同的有效路径，长短不一.
- **接口:**

py

```

1 @staticmethod
2 def generate_braid_maze(
3     rows: int, cols: int, braid_probability: float = 0.5
4 ) → Maze:
5     """生成编织迷宫（多解迷宫，存在环路）"""
6     ...

```

(4) generate_cave_maze: 生成**洞穴迷宫**（如 图 4 所示），即类似自然洞穴的迷宫。

- **原理:** 算法基于细胞自动机的思想.首先随机生成一个初始状态的网格，其中每个格子以一定概率被设为墙壁.然后，算法迭代地更新网格状态: 对于每个格子，统计其周围 8 个邻居中有多少是墙壁.如果邻居中墙壁数量超过某个阈值，则该格子在下一轮迭代中变成墙壁；反之则变成平地.经过多次迭代后，网格会逐渐演化成类似自然洞穴的结构.
- **地图特点:** 拥有大量不规则的开阔空间和狭窄通道，整体感觉更像是自然形成的洞穴系统，而不是人工设计的迷宫.
- **接口:**

py

```

1 @staticmethod
2 def generate_cave_maze(
3     rows: int, cols: int, initial_wall_prob: float = 0.45, iterations:
4     int = 4
5 ) → Maze:
6     """利用细胞自动机生成类似自然洞穴的迷宫"""
7     ...

```

(5) generate_recursive_division: 生成**递归分割迷宫**（如 图 5 所示），即拥有长直走廊和整齐的矩形区块的迷宫。

- **原理:** 算法采用分治的思想.首先将整个网格视为一个大矩形区域，在其中随机选择一个位置竖直或水平划分成两个子区域，并在划分线上随机打通一个洞作为通路.然后对每个子区域递归地执行同样的划分过程，直到子区域小到无法再划分为止.最终形成的迷宫由许多长直的走廊和整齐的矩形区块组成.
- **地图特点:** 拥有大量长直走廊和规整的矩形区块.它的结构相对简单，路径较为直接，但仍然存在一些死胡同和分叉.
- **接口:**

py

```

1 @staticmethod
2 def generate_recursive_division(rows: int, cols: int) → Maze:
3     """使用递归分割法生成迷宫（拥有长直走廊和整齐的矩形区块）"""
4     ...

```

(6) generate_prim_maze: 生成**Prim 迷宫**（如 图 6 所示），即极多超短死胡同，强烈的中心辐射感的迷宫。

- **原理:** 算法从起点开始，将其加入一个“已访问”集合.然后，算法将起点的所有邻居加入一个优先队列中，优先级根据某种随机权重来确定.每次从优先队列中取出优先级最高的邻居，如果该邻居未被访问过，则将其加入“已访问”集合，并打通它与它的父节点之间的墙.然后，将该邻居的所有未访问过的邻居加入优先队列中.这个过程持续进行，直到优先队列为空为止.

- **地图特点：**极多超短死胡同，强烈的中心辐射感.它的路径结构非常独特，中心区域通常会有更多的分叉和死胡同，而外围则相对简单.
- **接口：**

py

```
1 @staticmethod
2 def generate_prim_maze(rows: int, cols: int) → Maze:
3     """使用随机 Prim 算法生成迷宫（极多超短死胡同，强烈的中心辐射感）"""
4     ...
```

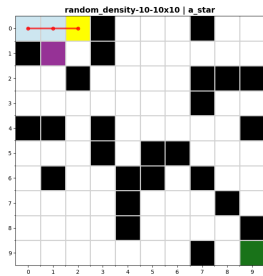


图 1 Random Density

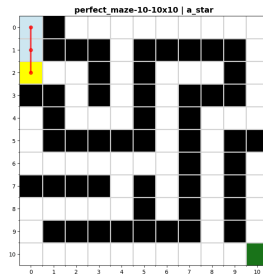


图 2 Perfect Maze

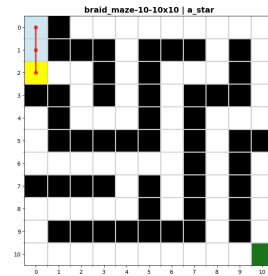


图 3 Braid Maze

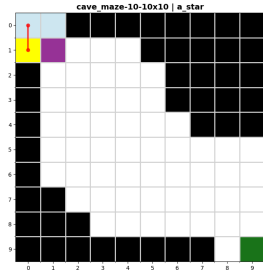


图 4 Cave Maze

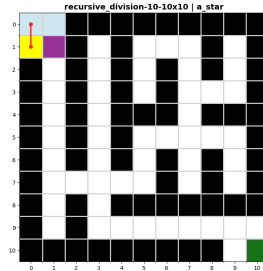


图 5 Recursive Division

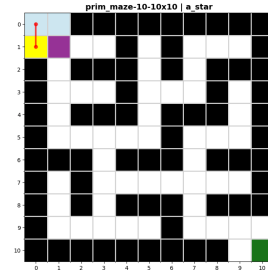


图 6 Prim Maze

III.3 搜索算法

在生成了不同类型的迷宫之后，我们需要实现多种搜索算法来寻找从起点到终点的最短路径.为此，我们定义了一个 `MazeSolver` 类，包含 DFS、BFS、Dijkstra 和 A* 四种算法的实现.每个算法都接受一个 `Maze` 实例作为输入，并返回一个 `SolverState` 的迭代器，方便后续的可视化展示.

- (1) **DFS：**深度优先搜索，维护一个栈，每次递归调用直到找到终点或无路可走时回退.

py

```
1 @staticmethod
2 def dfs(maze: Maze) → Iterator[SolverState]:
3     stack = [(maze.start, [maze.start])]
4     visited = set()
5     explored_order = []
6
7     while stack:
8         current, path = stack.pop()
9         if current in visited:
10             continue
11
12         visited.add(current)
13         explored_order.append(current)
14
15         # 提取栈中等待探索的节点
```

```

16     frontier = [node for (node, p) in stack if node not in visited]
17     yield SolverState(current, list(explored_order), frontier, path)
18
19     if current == maze.goal:
20         return
21
22     for nr, nc in maze.get_neighbors(*current):
23         if (nr, nc) not in visited:
24             stack.append(((nr, nc), path + [(nr, nc)]))

```

(2) **BFS**: 广度优先搜索, 维护一个队列, 每次探索当前层的所有节点后再进入下一层.

py

```

1 @staticmethod
2 def bfs(maze: Maze) → Iterator[SolverState]:
3     queue = deque([(maze.start, [maze.start])])
4     visited = {maze.start}
5     explored_order = []
6
7     while queue:
8         current, path = queue.popleft()
9         explored_order.append(current)
10
11         frontier = [node for (node, p) in queue]
12         yield SolverState(current, list(explored_order), frontier, path)
13
14         if current == maze.goal:
15             return
16
17         for nr, nc in maze.get_neighbors(*current):
18             if (nr, nc) not in visited:
19                 visited.add((nr, nc))
20                 queue.append(((nr, nc), path + [(nr, nc)]))

```

(3) **Dijkstra**: 维护一个优先队列, 每次选择当前路径代价最小的节点进行扩展.

py

```

1 @staticmethod
2 def dijkstra(maze: Maze) → Iterator[SolverState]:
3     # 优先队列存储: (当前累计代价, 当前坐标, 到达当前坐标的路径)
4     pq = [(0, maze.start, [maze.start])]
5     # 记录到达每个节点的最短已知代价
6     costs = {maze.start: 0}
7
8     explored_order = []
9     explored_set = set()
10
11     while pq:
12         current_cost, current, path = heapq.heappop(pq)
13
14         # 如果该节点已经被完全探索过, 说明队列里存放的是冗余的历史较高代价记录, 直接跳过
15         if current in explored_set:
16             continue
17
18         explored_set.add(current)
19         explored_order.append(current)
20
21         # 提取优先队列中还在等待探索的节点 (过滤掉已经处理完的)

```

```

22     frontier = list({node for cost, node, p in pq if node not in
23                     explored_set})
24     # yield 当前状态给可视化器
25     yield SolverState(current, list(explored_order), frontier, path)
26
27     if current == maze.goal:
28         return
29
30     for nr, nc in maze.get_neighbors(*current):
31         new_cost = current_cost + 1 # 迷宫中每走一步的代价视为 1
32
33         # 只有找到更优路径时, 才更新代价并加入队列
34         if new_cost < costs.get((nr, nc), float("inf")):
35             costs[(nr, nc)] = new_cost
36             heapq.heappush(pq, (new_cost, (nr, nc), path + [(nr,
37                                     nc)]))

```

Idea

实际上, 由于在这个迷宫问题中, 所有边的边权均为 1, 因此 Dijkstra 的**优先队列**本质上和 BFS 的普通队列是等价的. 因此在这个特定问题中, Dijkstra 和 BFS 的唯一的区别是 Dijkstra 维护了一个额外的成本字典来记录到达每个节点的最短已知代价, 而 BFS 则只需要维护一个简单的访问集合.

- (4) **A***: 在 Dijkstra 的基础上加入启发式函数 $f(n) = g(n) + h(n)$, 优先探索估价函数值最小的节点.

py

```

1  @staticmethod
2  def a_star(maze: Maze) → Iterator[SolverState]:
3      def manhattan(p1, p2):
4          return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
5
6      g_score = {maze.start: 0}
7      pq = [(manhattan(maze.start, maze.goal), maze.start, [maze.start])]
8      explored_order = []
9      explored_set = set()
10
11     while pq:
12         _, current, path = heapq.heappop(pq)
13
14         if current in explored_set:
15             continue
16
17         explored_set.add(current)
18         explored_order.append(current)
19         frontier = list({node for f, node, p in pq if node not in
20                         explored_set})
21
22         yield SolverState(current, list(explored_order), frontier, path)
23
24         if current == maze.goal:
25             return

```



```

26         for nr, nc in maze.get_neighbors(*current):
27             tentative_g = g_score.get(current, float("inf")) + 1
28             if tentative_g < g_score.get((nr, nc), float("inf")):
29                 g_score[(nr, nc)] = tentative_g
30                 f_score = tentative_g + manhattan((nr, nc), maze.goal)
31                 heapq.heappush(pq, (f_score, (nr, nc), path + [(nr, nc)]))

```

- (5) **A* with cross product**: 在 A* 的基础上加入向量叉积打破平局, 使得算法在开阔地形和复杂迷宫中都能表现出色.

Idea

注意到, 在开阔的地形, 比如 cave maze 和 random density maze 中, 一般的 A* 算法由于估价函数设计为曼哈顿距离, 因此会有很多“平局”的局面, 即多个节点的 $f(n)$ 值完全相同, 导致算法在这些节点之间无差别地进行扩展, 表现得像是 BFS 一样, 效率大打折扣.

此时, 我们希望在这些平局的局面中, 选择一个大方向更接近终点的节点, 也就是与终点方向的偏差距离最小. 形式化就是, 假设起点为 O , 终点为 G , 当前位置为 M , 则正确的方向应该是 $\vec{v}_0 = \overrightarrow{OG}$, 而现在的方向是 $\vec{v} = \overrightarrow{MG}$, 而偏差程度就可以使用 $|\vec{v}_0 \times \vec{v}|$ 来表征. 这个值越小, 说明当前节点 M 的方向与正确方向 OG 越接近, 我们就应该优先扩展它.

从而, 现在的启发函数就变为

$$f(M) = g(M) + \text{manhattan}(M, G) + \alpha \cdot (\overrightarrow{OG} \times \overrightarrow{MG})$$

其中, α 是一个非常小的系数, 保证它不会破坏 A* 的最优性, 但足以在平局时打破 BFS 式的无差别扩展, 让算法在开阔地形中表现得更像 A*, 而在复杂迷宫中仍然保持强大的寻路能力.

py

```

1 @staticmethod
2 def a_star_cross(maze: Maze) → Iterator[SolverState]:
3     """
4     高鲁棒性的 A* 算法: 使用向量叉积打破平局, 完美兼顾开阔地形与复杂迷宫.
5     """
6     def manhattan(p1, p2):
7         return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
8
9     g_score = {maze.start: 0}
10
11     # 提前获取起点和终点, 用于计算基础向量
12     sr, sc = maze.start
13     gr, gc = maze.goal
14
15     def heuristic(current):
16         cr, cc = current
17         h = manhattan(current, maze.goal)
18
19         # 【核心逻辑】: 计算向量叉积 (Cross Product)
20         # 衡量当前点到终点的向量, 偏离起点到终点“绝对直线”的程度
21         dx1 = cr - gr
22         dy1 = cc - gc

```



```

23     dx2 = sr - gr
24     dy2 = sc - gc
25     cross_product = abs(dx1 * dy2 - dx2 * dy1)
26
27     # 将叉积乘以一个极小的系数作为次要惩罚项
28     # 它小到不足以破坏寻路的最优性，但能完美打破开阔地面上的平局！
29     return h + cross_product * 0.001
30
31     pq = [(heuristic(maze.start), maze.start, [maze.start])]
32     explored_order = []
33     explored_set = set()
34
35     while pq:
36         _, current, path = heapq.heappop(pq)
37
38         if current in explored_set:
39             continue
40
41         explored_set.add(current)
42         explored_order.append(current)
43
44         frontier = list({node for f, node, p in pq if node not in
45                             explored_set})
46         yield SolverState(current, list(explored_order), frontier, path)
47
48         if current == maze.goal:
49             return
50
51         for nr, nc in maze.get_neighbors(*current):
52             tentative_g = g_score.get(current, float("inf")) + 1
53             if tentative_g < g_score.get((nr, nc), float("inf")):
54                 g_score[(nr, nc)] = tentative_g
55                 f_score = tentative_g + heuristic((nr, nc))
56                 heapq.heappush(pq, (f_score, (nr, nc), path + [(nr, nc)]))

```

- (6) **JPS 4-way**: 在 A* 的基础上加入跳点搜索的优化策略，极大减少在开阔地形中的冗余节点扩展。

Idea

JPS 是一种针对网格地图优化 A* 搜索的算法。它的核心思想是在长直走廊中直接“跳过”中间节点，只在路口（交汇点）或死胡同停下，从而大幅减少在开阔地形中的冗余节点扩展。

在四向网格中，跳点搜索的核心是一个“跳跃函数”，它接受当前坐标和一个移动方向作为输入，然后沿着这个方向不断前进，直到遇到以下情况之一：

- (1) 撞墙：如果前进过程中遇到墙壁，说明这条路不通，返回 None。
- (2) 找到终点：如果前进过程中达到了终点，直接返回终点坐标。
- (3) 遇到路口：如果在前进过程中发现当前方向的正交（垂直）方向上有可行走的邻居，说明这是一个路口，需要停下来进行扩展。

通过这种方式，它直接在路口和路口之间飞跃，直接把图的规模从“几万个像素格子”压缩成了“几十个关键路口构成的拓扑图”。

```

1 @staticmethod
2 def jps_4way(maze: Maze) → Iterator[SolverState]:
3     """
4     四向跳跃点搜索 (Junction Point Search / 4-way JPS)
5     在长直走廊中直接“跳过”中间节点，只在路口（交汇点）或死胡同停下。
6     """
7     def manhattan(p1, p2):
8         return abs(p1[0] - p2[0]) + abs(p1[1] - p2[1])
9
10    g_score = {maze.start: 0}
11    pq = [(manhattan(maze.start, maze.goal), maze.start, [maze.start])]
12    explored_order = []
13    explored_set = set()
14
15    def jump(r, c, dr, dc):
16        """核心跳跃函数：沿特定方向狂奔，直到遇到墙壁、终点或路口"""
17        path = []
18        while True:
19            nr, nc = r + dr, c + dc
20            if not maze.is_valid_move(nr, nc):
21                return None, path # 撞墙，此路不通
22
23            path.append((nr, nc))
24            if (nr, nc) == maze.goal:
25                return (nr, nc), path # 找到终点
26
27            # 【检测路口】：对于四向网格，如果当前移动方向是 (dr, dc)
28            # 那么它的正交（垂直）方向必定是 (dc, dr) 和 (-dc, -dr)
29            for odr, odc in [(dc, dr), (-dc, -dr)]:
30                if maze.is_valid_move(nr + odr, nc + odc):
31                    # 侧边有路可以走！这是一个路口 (Junction)，停止跳跃
32                    return (nr, nc), path
33
34            # 如果没有遇到路口，继续沿着原方向迭代跳跃
35            r, c = nr, nc
36
37    while pq:
38        _, current, path = heapq.heappop(pq)
39
40        if current in explored_set:
41            continue
42
43        # 【可视化适配】：由于 JPS 跳过了中间节点
44        # 为了让动画渲染出完整的光束，我们需要把跳跃经过的节点全部染色
45        for node in path:
46            if node not in explored_set:
47                explored_order.append(node)
48                explored_set.add(node)
49
50        frontier = list({node for f, node, p in pq if node not in explored_set})
51        yield SolverState(current, list(explored_order), frontier, path)
52
53        if current == maze.goal:
54            return

```

```

55
56     # 不再一格一格试探, 而是向四个方向发射“跳跃探测射线”
57     directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]
58     for dr, dc in directions:
59         if not maze.is_valid_move(current[0] + dr, current[1] + dc):
60             continue
61
62         jump_result, jump_path = jump(current[0], current[1], dr, dc)
63
64         if jump_result is not None:
65             # 代价不再是 +1, 而是要加上这次跳跃跨越的格数
66             tentative_g = g_score[current] + len(jump_path)
67
68             if tentative_g < g_score.get(jump_result, float("inf")):
69                 g_score[jump_result] = tentative_g
70                 f_score = tentative_g + manhattan(jump_result,
71                                                     maze.goal)
72                 # 将跳跃终点压入优先队列
73                 heapq.heappush(pq, (f_score, jump_result, path +
                                     jump_path))

```

III.4 可视化渲染

为了更直观地展示搜索算法的执行过程, 我们实现了一个 `MazeVisualizer` 类, 使用 `Matplotlib` 来动态渲染迷宫的状态. 每当搜索算法 `yield` 一个新的 `SolverState` 时, `MazeVisualizer` 就会更新图像, 显示当前节点、已探索的节点、待探索的节点以及当前路径.

py

```

1 class MazeVisualizer:
2     def __init__(self, maze: Maze):
3         self.maze = maze
4         self.fig, self.ax = plt.subplots(figsize=(10, 8))
5         self._setup_plot()
6
7         # 保存动态绘制对象的引用以便清除
8         self.dynamic_patches = {
9             "explored": [],
10            "frontier": [],
11            "current": [],
12            "goal": [],
13            "path": [],
14        }
15
16    def _setup_plot(self):
17        ...
18
19    def _clear_patches(self, key: str):
20        while self.dynamic_patches[key]:
21            self.dynamic_patches[key].pop().remove()
22
23    def _add_rect(self, r, c, color, alpha=1.0) → patches.Rectangle:
24        rect = patches.Rectangle(
25            (c - 0.5, r - 0.5), 1, 1, linewidth=0, facecolor=color, alpha=alpha
26        )
27        return self.ax.add_patch(rect)
28

```

```
29     def _update_frame(self, state: SolverState):
30         ...
31
32     def animate(
33         self, solver_generator: Iterator[SolverState], interval=100,
34         save_path=None
35     ):
36         ...
```

III.5 Benchmark

为了探究不同算法在不同类型的迷宫上的性能表现，我们设计了一套基准测试框架。我们生成了 1000 个不同随机种子下的迷宫实例，并在每个实例上运行所有算法，记录它们的路径长度、探索节点数和运行时间，计算平均值后结果整合在 [表 1](#) 中（见附录）。

我们可以得出以下结论：

(1) 盲目搜索的局限性与拓扑依赖 (DFS vs. BFS/Dijkstra)

- **最优性的丧失：**在存在环路的多解迷宫（Braid Maze）中，DFS 暴露了其无法保证最短路径的致命缺陷。以 128×128 的 Braid Maze 为例，BFS 和 A* 稳定找到平均长度为 524.7 的最短路径，而 DFS 找到的路径长度高达 1460.9，绕路极其严重。
- **Dijkstra 的退化：**数据表明，在所有地形下，Dijkstra 的探索节点数和路径长度均与 BFS 保持高度一致（例如 128×128 Perfect Maze 中探索数均为 4931.6 左右）。这是因为在本实验中迷宫步长权重统一为 1，Dijkstra 退化为普通的广度优先搜索，且由于优先队列的额外维护开销，其运行时间普遍略高于 BFS。

(2) 启发式搜索在开阔地形的“平局困境”与破局 (Standard A vs. A Plus)

- **A 的“胖菱形”膨胀：**在 Random Density 和 Cave Maze 这种拥有大片无障碍空地的地形中，标准的 A* 算法遭遇了严重的“平局”问题。由于多条路径的 $f(n)$ 估价完全相同，A* 被迫进行类似 BFS 的无差别扩展。在 128×128 的 Random Density 迷宫中，标准 A* 探索了多达 8113.3 个节点，耗时 248.30 毫秒。
- **叉积优化的降维打击：**引入向量叉积的 A* Plus 展现了极其恐怖的优化效果。同样在 128×128 的 Random Density 中，A* Plus 利用叉积惩罚完美打破平局，使得探索节点数骤降至 907.3（减少了约 89%），耗时仅 11.17 毫秒。
- **微小的最优性折损：**值得注意的是，在 128×128 Cave Maze 中，A* Plus 的平均路径长度为 256.4，略微高于标准最优解 256.3。这印证了理论推导：由于叉积项使得 $h(n)$ 产生了极小的高估，算法在换取百倍速度提升的同时，在极少数复杂边缘做出了微小的绕路妥协。

(3) 数据结构层面的空间跳跃优化 (JPS 4-Way 表现分析)

- JPS 展现了对具有长直走廊特性的地图的绝对统治力。在 Recursive Division 和 Perfect Maze 中，大量没有岔路的直线通道使得 JPS 能够直接“飞跃”中间节点。
- 以 128×128 的 Recursive Division 迷宫为例，标准 A* 需要探索 5549.8 个节点，而 JPS 仅需探索 3662.7 个关键路口，耗时大幅领先。在 Prim Maze 这种充满极其短小死胡同的“高岔路”地形中，JPS 依然能通过快速的直线探测有效剪枝，探索数 (1576.6) 显著低于 A* Plus (2087.9)。

IV 总结

本项目以经典二维网格寻路问题为切入点，不仅成功实现了基础的 DFS、BFS、Dijkstra 和 A* 算法，更在系统架构和算法深度优化上进行了全方位的探索：

- (1) **工程架构的高内聚低耦合：** 我们设计了 Maze 实体类、MazeGenerator 地图生成器和 MazeSolver 算法工厂.尤其是在算法实现中，利用 Python 的 yield 特性构建了 SolverState 的生成器模式，将底层寻路逻辑与前端 MazeVisualizer 动画渲染完美解耦。
- (2) **多样化测试环境的构建：** 为了全面评估算法的鲁棒性，我们利用随机深度优先、细胞自动机、递归分割等原理，实现了多达 6 种具有显著不同拓扑结构特征的迷宫生成算法。这使得算法的优劣势（如 DFS 的深陷死胡同、BFS 在开阔地形的爆炸）暴露无遗。
- (3) **高阶算法的探索与论证：** 在基准测试中，我们不满足于标准的 A*，针对开阔地形创新性地引入了基于向量叉积的启发函数微调（A* Plus），以及针对长直走廊的四向跳跃点搜索（JPS 4-Way）。1000 次蒙特卡洛平均测试数据有力地证明了：没有绝对完美的寻路算法，最高效的搜索永远建立在对物理拓扑结构的充分利用之上。

i Info

完整代码见压缩包其余代码文件。

V 附录

Size	Maze	Algorithm	Path Len	Explored	Time (ms)
8 × 8	Random Density	DFS	27.6	34.2	0.05
		BFS	15.0	50.2	0.06
		Dijkstra	15.0	50.4	0.09
		A*	15.0	41.4	0.09
		A* Plus ¹	15.0	21.5	0.05
		JPS (4-Way)	15.0	39.8	0.12
	Braid Maze	DFS	28.4	29.2	0.03
		BFS	27.4	35.4	0.04
		Dijkstra	27.4	35.5	0.05
		A*	27.4	30.4	0.05
		A* Plus	27.4	29.8	0.05
		JPS (4-Way)	27.4	29.1	0.04
	Perfect Maze	DFS	28.1	28.2	0.03
		BFS	28.1	33.0	0.03
		Dijkstra	28.1	33.0	0.04
		A*	28.1	29.6	0.05
		A* Plus	28.1	29.3	0.05
		JPS (4-Way)	28.1	28.6	0.04
	Cave Maze	DFS	27.7	31.7	0.05

¹这里的算法指的是 A* with cross product

Size	Maze	Algorithm	Path Len	Explored	Time (ms)
16 × 16		BFS	15.0	38.3	0.04
		Dijkstra	15.0	38.3	0.06
		A*	15.0	37.6	0.08
		A* Plus	15.0	20.4	0.05
		JPS (4-Way)	15.0	37.6	0.11
		DFS	17.5	24.4	0.03
		BFS	17.5	34.9	0.04
		Dijkstra	17.5	35.3	0.06
		A*	17.5	26.2	0.05
		A* Plus	17.5	22.5	0.05
		JPS (4-Way)	17.5	22.2	0.05
	Prim Maze	DFS	18.5	22.2	0.03
		BFS	18.5	34.3	0.04
		Dijkstra	18.5	34.6	0.05
		A*	18.5	27.2	0.05
		A* Plus	18.5	23.8	0.06
		JPS (4-Way)	18.5	23.2	0.05
	Recursive Division	DFS	71.7	107.6	0.29
		BFS	31.1	203.2	0.28
		Dijkstra	31.1	203.4	0.46
		A*	31.1	147.1	0.46
		A* Plus	31.1	57.9	0.18
		JPS (4-Way)	31.1	142.1	0.59
	Random Density	DFS	73.4	84.8	0.11
		BFS	64.9	119.1	0.15
		Dijkstra	64.9	119.6	0.20
		A*	64.9	95.4	0.18
		A* Plus	64.9	91.4	0.18
		JPS (4-Way)	64.9	87.3	0.14
	Braid Maze	DFS	76.9	78.9	0.10
		BFS	76.9	102.3	0.12
		Dijkstra	76.9	102.3	0.16
		A*	76.9	90.6	0.16
		A* Plus	76.9	89.5	0.17
		JPS (4-Way)	76.9	83.8	0.13
	Perfect Maze	DFS	95.9	118.0	0.35
		BFS	31.0	148.7	0.20
		Dijkstra	31.0	148.9	0.31

Size	Maze	Algorithm	Path Len	Explored	Time (ms)
32 × 32	Prim Maze	A*	31.0	132.3	0.37
		A* Plus	31.0	51.5	0.16
		JPS (4-Way)	31.0	132.1	0.52
		DFS	35.3	71.6	0.09
		BFS	35.3	121.3	0.15
		Dijkstra	35.3	122.1	0.22
		A*	35.3	67.9	0.14
		A* Plus	35.3	56.8	0.13
		JPS (4-Way)	35.3	52.3	0.11
	Recursive Division	DFS	42.4	69.5	0.09
		BFS	42.4	115.4	0.14
		Dijkstra	42.4	116.0	0.20
		A*	42.4	82.8	0.16
		A* Plus	42.4	75.6	0.16
		JPS (4-Way)	42.4	63.6	0.12
	Random Density	DFS	180.9	366.1	2.58
		BFS	63.1	815.6	1.48
		Dijkstra	63.1	815.7	2.62
		A*	63.1	541.1	2.62
		A* Plus	63.1	146.8	0.61
		JPS (4-Way)	63.1	524.1	3.12
	Braid Maze	DFS	196.5	291.3	0.49
		BFS	141.4	458.4	0.71
		Dijkstra	141.4	459.8	0.99
		A*	141.4	351.7	0.83
		A* Plus	141.4	339.0	0.84
		JPS (4-Way)	141.4	313.9	0.65
	Perfect Maze	DFS	227.9	243.0	0.36
		BFS	227.9	348.2	0.52
		Dijkstra	227.9	348.2	0.65
		A*	227.9	313.4	0.66
		A* Plus	227.9	310.6	0.70
		JPS (4-Way)	227.9	283.4	0.61
	Cave Maze	DFS	274.7	410.7	3.00
		BFS	63.2	611.3	1.03
		Dijkstra	63.2	612.1	1.75
		A*	63.2	450.1	1.92
		A* Plus	63.2	167.7	0.65

Size	Maze	Algorithm	Path Len	Explored	Time (ms)
64 × 64	Prim Maze	JPS (4-Way)	63.2	449.6	2.56
		DFS	72.4	246.3	0.41
		BFS	72.4	466.6	0.72
		Dijkstra	72.4	468.2	1.16
		A*	72.4	212.6	0.60
		A* Plus	72.4	177.7	0.50
		JPS (4-Way)	72.4	148.8	0.36
		DFS	101.2	247.6	0.42
		BFS	101.2	433.1	0.66
		Dijkstra	101.2	434.4	0.96
		A*	101.2	318.8	0.80
		A* Plus	101.2	303.4	0.82
		JPS (4-Way)	101.2	223.1	0.52
	Random Density	DFS	411.2	1211.4	31.66
		BFS	127.1	3266.3	15.61
		Dijkstra	127.1	3266.4	22.07
		A*	127.1	2064.7	22.30
		A* Plus	127.1	386.3	2.84
		JPS (4-Way)	127.1	2002.2	23.94
	Braid Maze	DFS	536.1	1080.8	3.78
		BFS	277.7	1971.7	6.17
		Dijkstra	277.7	1974.5	8.24
		A*	277.7	1523.6	6.29
		A* Plus	277.7	1484.9	6.24
		JPS (4-Way)	277.7	1350.7	4.19
	Perfect Maze	DFS	738.8	838.6	2.15
		BFS	738.8	1297.8	3.99
		Dijkstra	738.8	1297.9	4.57
		A*	738.8	1208.2	4.49
		A* Plus	738.8	1202.9	4.61
		JPS (4-Way)	738.8	1077.7	4.60
	Cave Maze	DFS	724.0	1450.9	33.93
		BFS	128.2	2579.9	10.12
		Dijkstra	128.2	2581.3	14.97
		A*	128.2	1491.3	12.62
		A* Plus	128.2	621.8	4.01
		JPS (4-Way)	128.2	1489.7	15.64
	Prim Maze	DFS	147.1	887.5	2.30

Size	Maze	Algorithm	Path Len	Explored	Time (ms)
128 × 128		BFS	147.1	1820.5	5.56
		Dijkstra	147.1	1824.2	8.28
		A*	147.1	720.1	3.02
		A* Plus	147.1	616.3	2.55
		JPS (4-Way)	147.1	472.9	1.42
		DFS	248.3	939.1	2.60
		BFS	248.3	1680.9	4.72
		Dijkstra	248.3	1683.2	6.31
		A*	248.3	1305.2	5.03
		A* Plus	248.3	1275.9	5.07
		JPS (4-Way)	248.3	876.3	2.77
	Random Density	DFS	879.3	4448.5	512.95
		BFS	255.0	13074.5	226.21
		Dijkstra	255.0	13074.6	271.44
		A*	255.0	8113.3	248.30
		A* Plus	255.0	907.3	11.17
		JPS (4-Way)	255.0	7875.0	245.41
	Braid Maze	DFS	1460.9	3853.1	45.52
		BFS	524.7	8330.2	92.57
		Dijkstra	524.7	8333.8	105.79
		A*	524.7	6846.1	81.48
		A* Plus	524.7	6615.2	77.81
		JPS (4-Way)	524.7	6054.9	38.87
	Perfect Maze	DFS	2365.9	2902.2	22.52
		BFS	2365.9	4931.6	49.80
		Dijkstra	2365.9	4931.7	52.38
		A*	2365.9	4699.5	50.01
		A* Plus	2365.9	4682.5	50.28
		JPS (4-Way)	2365.9	4164.0	50.14
	Cave Maze	DFS	1964.8	5324.1	416.34
		BFS	256.3	10860.6	154.41
		Dijkstra	256.3	10861.6	187.43
		A*	256.3	5120.1	106.13
		A* Plus	256.4	1954.3	24.80
		JPS (4-Way)	256.3	5115.3	122.33
	Prim Maze	DFS	295.0	3444.7	24.68
		BFS	295.0	7153.9	72.08
		Dijkstra	295.0	7162.3	87.75

Size	Maze	Algorithm	Path Len	Explored	Time (ms)
		A*	295.0	2484.1	21.40
		A* Plus	295.0	2087.9	16.54
		JPS (4-Way)	295.0	1576.6	7.06
		DFS	630.0	3654.2	30.22
		BFS	630.0	6663.8	63.08
	Recursive	Dijkstra	630.0	6667.8	71.64
	Division	A*	630.0	5549.8	55.75
		A* Plus	630.0	5461.9	55.01
		JPS (4-Way)	630.0	3662.7	22.44

表 1 1000 次采样平均值结果